AgroScan 1.0 (Launching) - Server.js Documentation

Author: Abhinav Ranjan
Company: AgroScan Development by Luminary Developers

Description: This document details the endpoints, logic, and working of the `server.js` file for the AgroScan AI system.

---

1. Core Imports and Setup

- **Express:** HTTP server framework.
- **dotenv:** Loads environment variables.
- **body-parser:** Parses JSON requests.
- **cors:** Enables Cross-Origin Resource Sharing.
- **multer:** Handles file uploads.
- **mysql2/promise:** MySQL connection pool.
- **bcrypt:** Password hashing (for developer users).
- **child_process (exec):** Execute server commands.
- **os:** System statistics.

Local modules: - `ruleEngine.js` - Rule-based recommendation logic. - `perplexity-recommend-handler.js` - Perplexity AI recommendation. - `perplexity-chat-handler.js` - Perplexity AI chat integration.

Environment Variables: - `PORT` - server port, default `3000`. - `AI_PROVIDER` - AI provider, default `openai`.

---

2. Middleware

- `cors()` - enables cross-origin requests.
- `bodyParser.json()` - parses incoming JSON.
- `multer.memoryStorage()` - stores uploaded files in memory.

---

3. Database Connection

- MySQL connection pool using `mysql2/promise`.
- Credentials fetched from `.env`.

---

4. Health Check and Stats Endpoints

4.1 Root Endpoint `/`

- Returns server running status and active AI provider.

4.2 `/test`

- Returns `{ status: 'Server is running!' }`

4.3 `/api/server/stats`

- Returns system stats:
  - Memory: rss, heapTotal, heapUsed, arrayBuffers, totalMem, freeMem
  - CPU: cores, model, speed, loadAvg
  - Uptime, platform, architecture, Node version

---

5. User Management Endpoints

5.1 GET User Profile `/api/users/:userId`

- Returns user details (excluding password).

- Adds `profileUrl` endpoint link.

5.2 PUT Update Primary Info `/api/users/:userId`

- Updates `name`, `username`, `email`, `phone`.
- Handles duplicate entry errors.

5.3 PUT Update Address `/api/users/:userId/address`

- Updates `street`, `city`, `state`, `zip`, `country`.

5.4 PUT Change Password `/api/users/:userId/change-password`

- Plain-text comparison of old password.
- Updates password directly in DB.
- Returns success or error messages.

5.5 DELETE User `/api/users/:userId`

- Deletes user in a transaction.
- Also deletes related `lands` entries.
- Ensures rollback on error.

5.6 POST Login `/api/login`

- Authenticates by `username`, `email`, or `phone`.
- Plain-text password verification.
- Returns user details on success.

5.7 POST Signup `/api/signup`

- Creates a new user.
- Requires `name`, `username`, `email`, `phone`, `password`.
- Handles duplicates with `409 Conflict`.

---

6. User Desk - Lands Management

6.1 GET All Lands `/api/users/:userId/lands`

- Returns all lands for a user ordered by `created_at DESC`.

6.2 GET Single Land `/api/users/:userId/lands/:landId`

- Returns single land details.
- Checks ownership with `user_id`.

6.3 POST Add Land `/api/users/:userId/lands`

- Requires `land_name` and `size`.
- Optional fields default to `null`.
- Inserts into `lands` table with `created_at = NOW()`.

6.4 PUT Update Land `/api/users/:userId/lands/:landId`

- Updates existing land.
- Checks ownership.
- Optional fields default to `null`.

6.5 DELETE Land `/api/users/:userId/lands/:landId`

- Deletes specific land.
- Checks ownership.

---

7. User Desk - Orders

7.1 GET Orders `/api/users/:userId/orders`

- Returns all orders for a user.
- Ordered by `created_at DESC`.

7.2 POST New Order `/api/users/:userId/orders`

- Accepts array of items with `name`, `qty`, `price`.

- Optional `address`, `payment_method`, `coupon_code`, `name`, `phone`.
- Inserts each item in a transaction.
- Calculates `item_total = qty * price`.
- Rolls back on error.

---

8. Recommendation Endpoint `/recommend`

- Accepts `ph`, `moisture`, `temperature`, `location`, `desiredCrop`.
- Validates essential fields `ph` and `moisture`.
- Calls `ruleEngine()` for local logic.
- Calls `askPerplexityRecommend()` for AI recommendations.
- Returns combined result.

---

9. Chat Endpoints

9.1 POST `/chat`

- Accepts `message` and `userId`.
- Fetches or creates chat session (`chat_sessions` table).
- Stores message in `ai_chats` table.
- Retrieves last 5 messages for context.
- Calls `askPerplexityChat()`.
- Stores AI response in DB and returns it.

9.2 GET Chat History `/api/chat/history/:userId`

- Returns paginated chat history.
- Default `limit=10`, `offset=0`.
- Includes `hasMore` for pagination.

9.3 DELETE `/api/chat/clear/:userId`

- Deletes all chat messages for a user.
- No session management.

---

10. Profile Image Endpoints

10.1 GET `/api/users/:userId/profile`

- Retrieves user profile image.
- Streams as `image/jpeg`.
- Returns `404` if not set.

10.2 POST `/api/users/:userId/profile`

- Uploads and stores image as binary in `users.profile`.
- Expects `multipart/form-data` with `profile` field.
- Returns updated `profileUrl`.

---

11. Controls Management Endpoints

11.1 GET All `/api/controls`

- Returns all system controls.

11.2 GET Single `/api/controls/:function_name`

- Returns specific control.

11.3 POST Add `/api/controls`

- Inserts new control.
- Validates `function_name` and `value`.

11.4 PUT Update `/api/controls/:function_name`

- Updates `value` of existing control.
- Returns `404` if not found.

11.5 DELETE `/api/controls/:function_name`

- Deletes specific control.

---

12. Developer Desk

12.1 Dev Login `/api/dev-login`

- Auth via `email` and plain-text password.
- Returns dev user info.

12.2 Dev Login via UDC `/api/dev-login-udc`

- Auth via `uic` (QR/UDC code).
- Returns dev user info.

12.3 Execute Raw SQL `/api/sql-run`

- Executes arbitrary SQL.
- No sanitization.
- Returns query results.

12.4 DevUsers CRUD

- GET `/api/devusers/:id` - fetch dev user.
- PUT `/api/devusers/:id` - update profile.
- PUT `/api/devusers/:id/change-password` - supports bcrypt or plain-text variants.

12.5 Dev Tables `/api/dev/tables`

- Returns first 20 rows of all tables.
- Useful for debugging.

---

13. Deployment

- `app.listen(PORT)` starts the server.
- Logs server URL and active AI provider.

Notes on Logic

1. **Plain-text passwords** used for production users - not secure; recommend hashing.
2. **Transactions** used for user deletion and multi-item order insertion.
3. **Ownership checks** enforced for lands CRUD.
4. **RuleEngine + AI** combination provides local + AI-based recommendations.
5. **Chat sessions** maintained to contextualize AI responses.
6. **Profile images** stored as binary buffers in DB.
7. **Controls table** acts as system-wide feature/config flags.
8. **Developer endpoints** allow high-level DB access and debugging; caution advised.
